



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

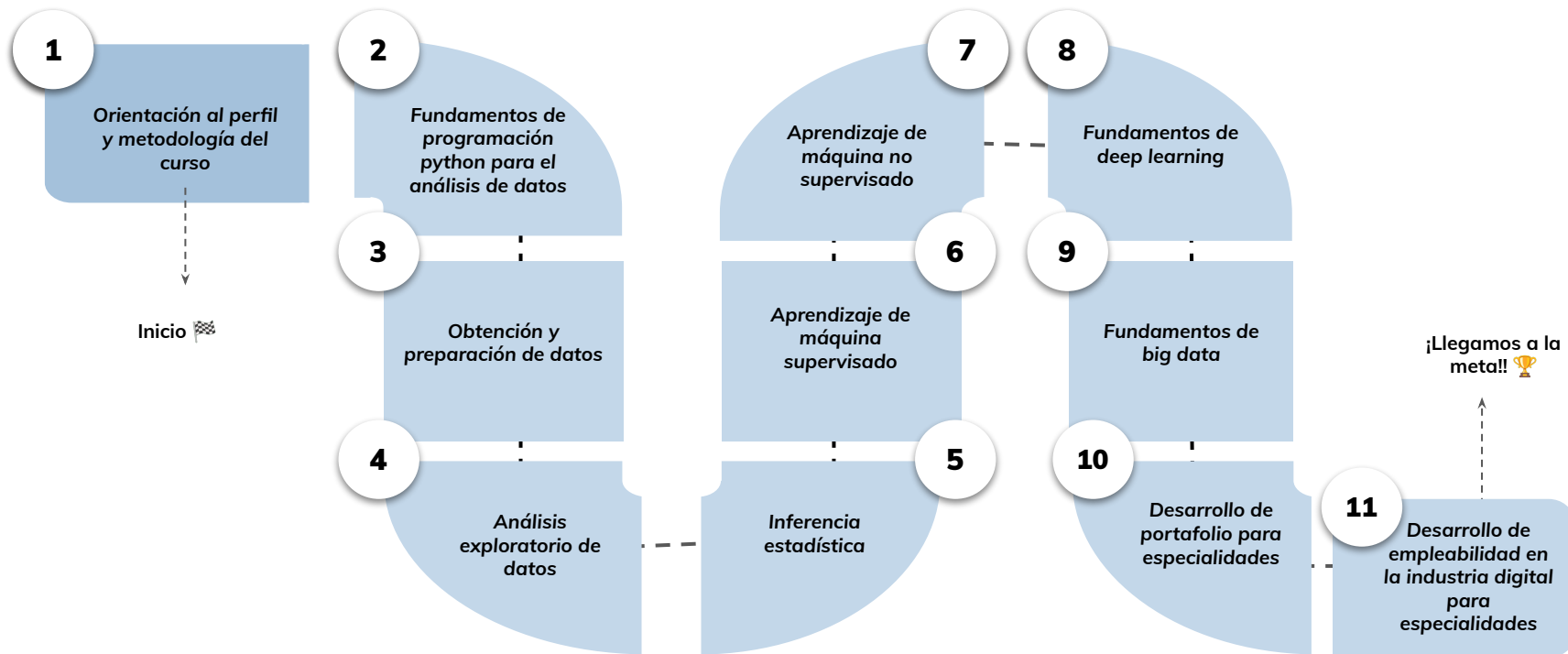


› Implementación de rn en python- Parte II

Aprendizaje Esperado 3: Implementar una red neuronal artificial utilizando python para resolver un problema de regresión y/o clasificación.

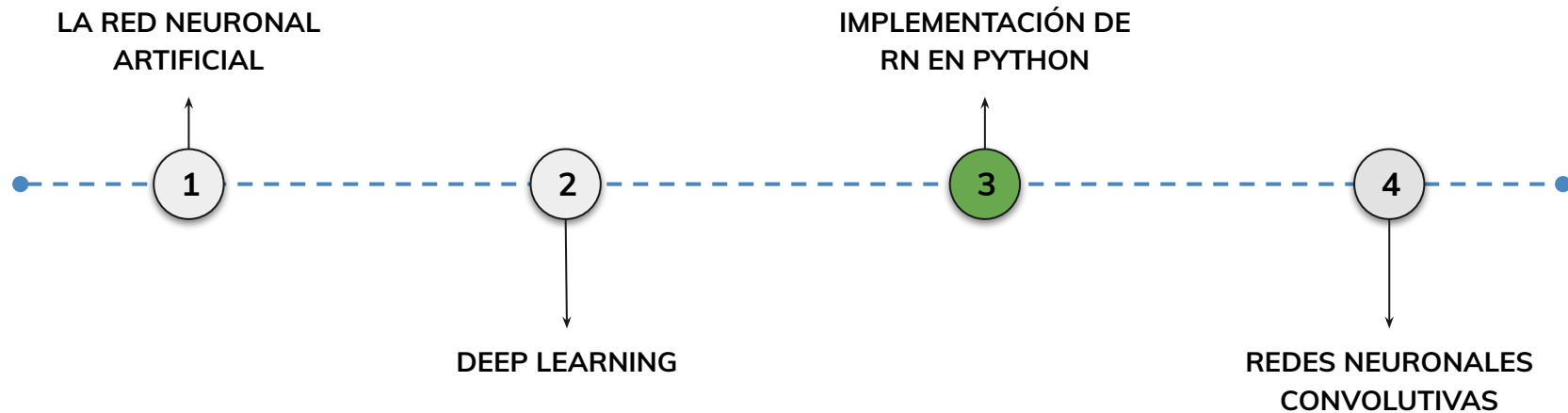
Hoja de ruta

¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



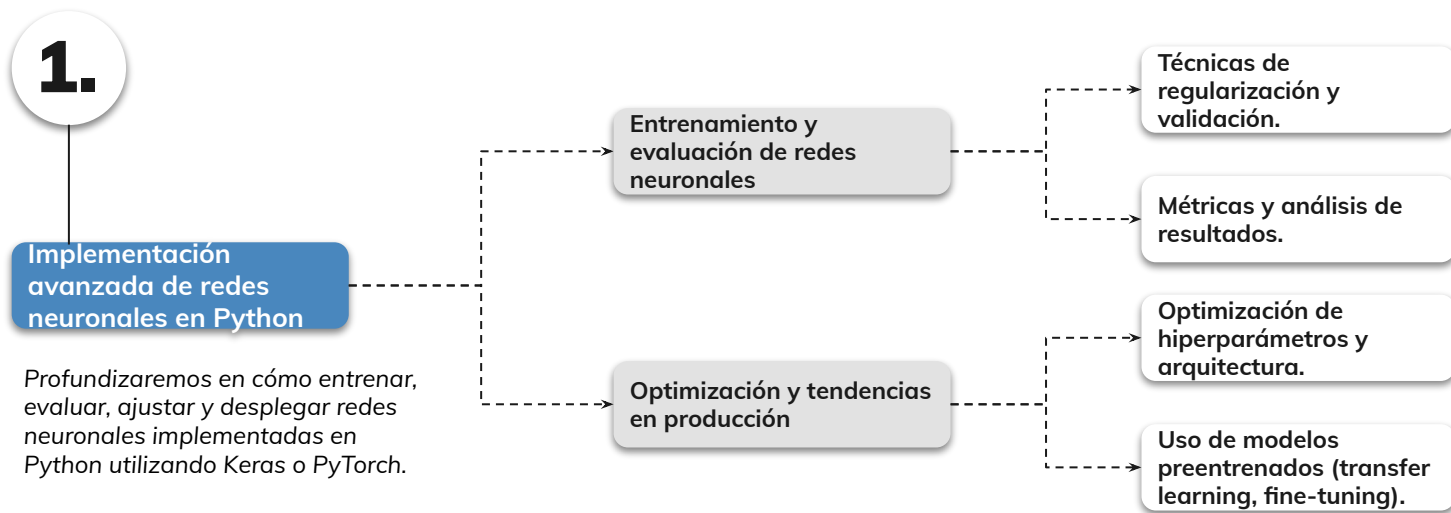
Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?





Objetivos de aprendizaje

¿Qué aprenderás?



- Entrenar, evaluar y ajustar redes neuronales en Python.
- Generar predicciones confiables con modelos ya entrenados.
- Implementar técnicas de regularización y validación avanzadas.
- Conocer estrategias de transfer learning y optimización de modelos.
- Reconocer los pasos iniciales para llevar un modelo a producción.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Identificamos los entornos de desarrollo en Python (Keras y PyTorch) para implementar redes neuronales.
- Comprendimos el concepto de tensores y cómo son la base del flujo de datos.
- Diseñamos arquitecturas básicas de redes para regresión y clasificación.
- Vimos funciones de activación, funciones de pérdida y optimizadores comunes.

Implementación de rn en python

Dropout en Keras

Esto mejora la generalización y reduce el riesgo de sobreentrenamiento.



Realizando Predicciones con el Modelo Neuronal

Una vez entrenado y evaluado, el modelo puede ser utilizado para predecir sobre nuevos datos con el método `.predict()` (Keras) o el método `.forward()` seguido de `.argmax()` en PyTorch.



Ejemplo de Predicción

```
predicciones = modelo.predict(nuevos_data)
```

Las predicciones pueden ser probabilísticas (probabilidad de pertenecer a una clase) o determinísticas (clase predicha).

Accesibilidad de las Redes Neuronales

El desarrollo e implementación de redes neuronales en Python representa una puerta de entrada accesible y poderosa al mundo del aprendizaje profundo.





Ventajas de las Bibliotecas Modernas

Gracias a bibliotecas como Keras y PyTorch, hoy es posible construir modelos efectivos con pocas líneas de código y enfocarse en el análisis de datos y la lógica del problema en lugar de en la infraestructura técnica.

Beneficios de Dominar estos Entornos

Mayor productividad

Desarrollo rápido de modelos complejos.

Experimentación ágil

Prueba de distintas configuraciones.

Versatilidad

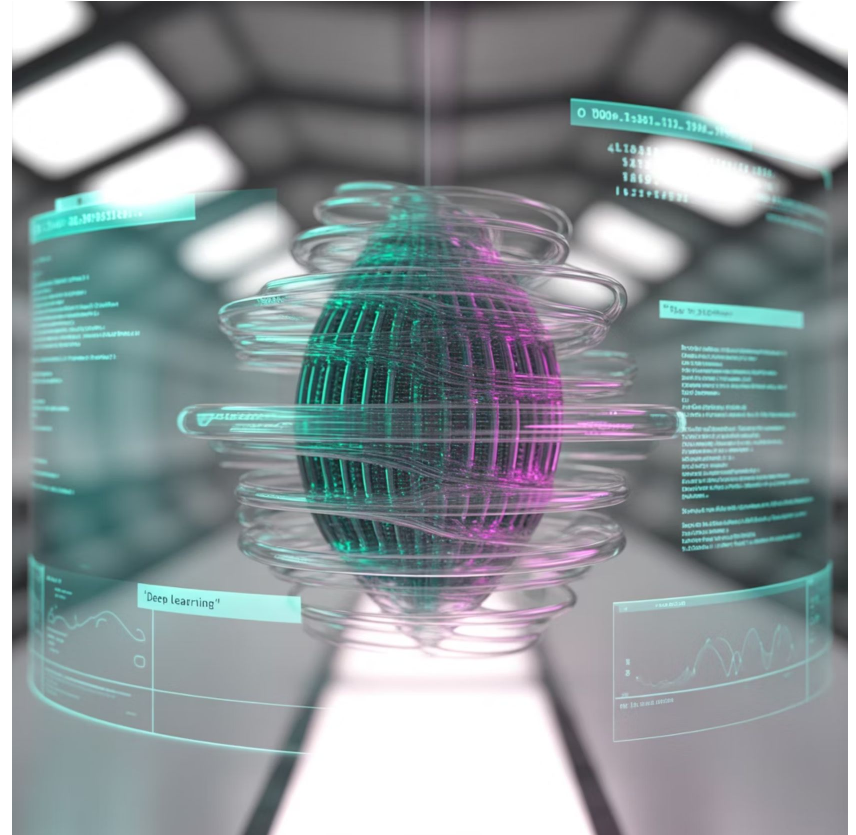
Adaptación a diferentes tareas y datos.

Elementos Principales de una Red Neuronal

A lo largo de este manual se exploraron los principales elementos que intervienen en la construcción de una red neuronal:

Comprensión de los Tensores

Los tensores son la unidad básica de datos en deep learning, permitiendo representar información de diferentes dimensiones.



Diseño de Arquitecturas

Clasificación

Redes diseñadas para asignar etiquetas a los datos de entrada.

Regresión

Redes orientadas a predecir valores numéricos continuos.

Entrenamiento de Redes Neuronales

Funciones de pérdida

Miden el error de las predicciones.

Herramientas de evaluación

Validan el desempeño del modelo.



Optimizadores

Ajustan los pesos de la red.

Técnicas de regularización

Previenen el sobreajuste.

Herramientas de Evaluación y Predicción

Claves para validar el desempeño del modelo y asegurar que pueda generalizar correctamente a nuevos datos.

Aplicaciones Prácticas

Contar con estas habilidades te permitirá implementar soluciones concretas a problemas reales en diversos campos.

Campos de Aplicación



Salud

Diagnóstico médico, análisis de imágenes, predicción de enfermedades.



Finanzas

Detección de fraude, predicción de mercados, evaluación de riesgos.



Automatización

Control de procesos, mantenimiento predictivo, robótica.



Análisis de texto

Procesamiento del lenguaje natural, clasificación de documentos, chatbots.

El Comienzo de un Camino

Este es solo el comienzo: con la base adquirida, podrás continuar explorando arquitecturas más complejas, aplicar modelos preentrenados y optimizar redes para producción en entornos reales.



Arquitecturas Avanzadas

Redes Convolucionales (CNN)

Especializadas en procesamiento de imágenes.

Redes Recurrentes (RNN)

Para datos secuenciales como texto o series temporales.

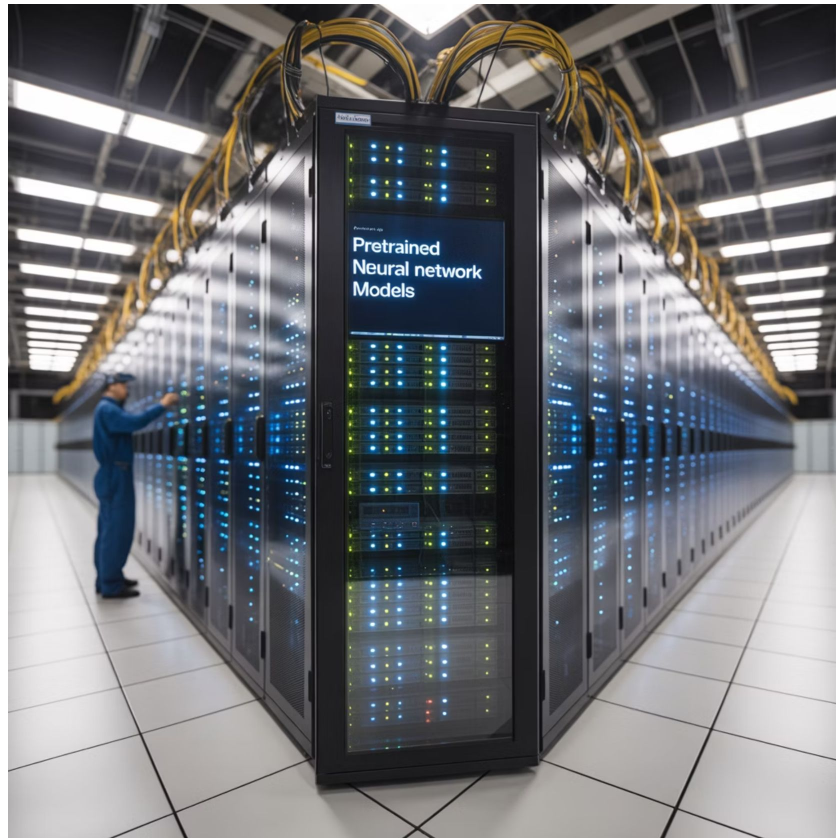
Transformers

Arquitectura avanzada para procesamiento de lenguaje natural.

Modelos Preentrenados

Utilizar modelos ya entrenados en grandes conjuntos de datos puede ahorrar tiempo y recursos.

- Transfer learning
- Fine-tuning
- Feature extraction



Optimización para Producción



Mejora de rendimiento

Optimización de hiperparámetros y arquitectura.



Eficiencia computacional

Reducción de tamaño y complejidad del modelo.



Despliegue

Integración en sistemas y aplicaciones reales.

Principales Desafíos

Datos insuficientes

Limitaciones en la cantidad o calidad de datos disponibles.

Sobreajuste

Modelos que memorizan en lugar de generalizar.

Recursos computacionales

Necesidad de hardware especializado para modelos complejos.

Interpretabilidad

Dificultad para entender las decisiones del modelo.

Estrategias para Superar los Desafíos

Aumento de datos

Generar variaciones sintéticas de los datos existentes.

Regularización

Aplicar técnicas como dropout y L2 para prevenir el sobreajuste.

Computación distribuida

Utilizar múltiples GPUs o TPUs para acelerar el entrenamiento.

Modelos explicables

Desarrollar técnicas para interpretar las decisiones del modelo.

Principales Tendencias



Eficiencia energética

Modelos que requieren menos recursos computacionales.



Aprendizaje federado

Entrenamiento distribuido que preserva la privacidad.



Computación neuromórfica

Hardware inspirado en el cerebro humano.



Modelos multimodales

Integración de diferentes tipos de datos (texto, imagen, audio).

Consideraciones Éticas

El desarrollo de redes neuronales plantea importantes cuestiones éticas que deben ser consideradas:

- Sesgos en los datos y modelos
- Privacidad y seguridad
- Impacto social y laboral
- Transparencia y responsabilidad



Recursos para Continuar Aprendiendo

Cursos en línea

Plataformas como Coursera, edX, Udacity ofrecen cursos especializados.

Libros

"Deep Learning" de Goodfellow, Bengio y Courville es una referencia fundamental.

Comunidades

Participar en foros como Stack Overflow, Reddit r/MachineLearning o Kaggle.

Proyectos prácticos

Implementar modelos para resolver problemas reales o participar en competencias.

Herramientas Complementarias



Jupyter Notebooks

Entorno interactivo ideal para experimentación y visualización.



Control de versiones

Git y GitHub para gestionar el código y colaborar.



Plataformas cloud

Google Colab, AWS, Azure para acceder a recursos computacionales.



Herramientas de visualización

Matplotlib, Seaborn, TensorBoard para analizar resultados.

Consejos para Principiantes

- Comienza con problemas sencillos
- Utiliza datasets públicos bien documentados
- Experimenta con modelos preentrenados
- Participa en comunidades de aprendizaje
- Practica regularmente con proyectos pequeños
- No te desanimes por los errores iniciales



Conclusión

Las redes neuronales en Python representan una poderosa herramienta para resolver problemas complejos en diversos campos. Con los conocimientos adquiridos en este manual, estás preparado para comenzar tu viaje en el fascinante mundo del aprendizaje profundo.

Recuerda que la práctica constante y la experimentación son claves para dominar estas técnicas. ¡El futuro de la inteligencia artificial está en tus manos!



Live Coding

¿En qué consistirá la Demo?

Explicaremos los pasos conceptuales para implementar una red neuronal artificial en Python resolviendo un problema de regresión y otro de clasificación, enfocándonos en el proceso de entrenamiento y evaluación.

1. Preparar datos para entrenar una red neuronal (train/test/validación).
2. Diseñar una red neuronal ajustada al problema (capas, funciones de activación).
3. Seleccionar métricas adecuadas: MSE, MAE (regresión) y Accuracy, F1 (clasificación).
4. Aplicar técnicas de regularización como dropout y early stopping.
5. Generar predicciones y analizar su interpretación (probabilidades vs valores continuos).
6. Discutir cómo optimizar la arquitectura y los hiperparámetros.



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Entrena y ajusta tu red neuronal

Entrena y ajusta tu red neuronal

Contexto: 🙌

Aplicaremos el diseño de redes neuronales visto en la Parte 1 para entrenarlas, evaluarlas y optimizarlas en Python.

Consigna: 📝

1. Selecciona un dataset (puede ser el mismo de la Parte 1 o uno nuevo).
2. Entrena tu red neuronal para resolver el problema (regresión o clasificación).
3. Evalúa el desempeño del modelo con métricas adecuadas.
4. Aplica al menos dos técnicas de mejora (regularización, early stopping, ajuste de hiperparámetros).
5. Genera predicciones y analiza si el modelo generaliza bien.

Paso a paso: ⚙️

1. Ajusta la arquitectura y los hiperparámetros según la complejidad del problema.
2. Evalúa el modelo en datos no vistos.
3. Documenta las decisiones tomadas y los resultados obtenidos.

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Entrenamos y evaluamos redes neuronales implementadas en Python.
- ✓ Aplicamos técnicas de regularización y validación para mejorar la generalización.
- ✓ Ajustamos hiperparámetros y analizamos métricas para regresión y clasificación.
- ✓ Generamos predicciones con redes neuronales entrenadas.



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

